DEPI · MICROSOFT MACHINE LEARNING ENGINEER TRACK

Land Type Classification

Using Sentinel-2 Satellite Images · EuroSAT Deep Learning Pipeline

PROJECT TYPE
AI / Computer Vision · Deep Learning

INTERNSHIP TRACK
DEPI – Microsoft ML Engineer

DATASET
EuroSAT · RGB + 13-Band TIF

CORE ARCHITECTURE
ResNet-50 · Transfer Learning

TEAM MEMBERS

Abdalaziz Atef Mohammed ★ LEADER

Fatma Osama Abdalaal

Abdallah Ashraf Mohammed

Maram Mahmoud Mohammed

Abdalrahman Mamdouh Rabia

Sarah Hagag Mohammed

PROJECT REPOSITORY

⌥github.com/Abdalaziz-AlHakim/Land-Type-Classification-Using-Sentinel2-Satellite-Images

🗒 Executive Summary

This project focuses on developing a machine learning system for automated land type classification using satellite or aerial imagery. The system aims to classify land into predefined categories such as urban areas, agricultural land, water bodies, forests, and barren land. The solution leverages computer vision and deep learning techniques and follows the full machine learning lifecycle, from data preprocessing to deployment and evaluation.

🎯 Project Objectives

- Build a reliable land type classification model.
- Apply supervised learning using labeled land imagery.
- Evaluate model performance using clear metrics.
- Deploy a reproducible and scalable ML pipeline.
- Ensure equal contribution across all team members.

❓ Problem Definition & Research Question

Research Question: Can a machine learning model accurately classify land types from satellite imagery within the given project timeline?

SIGNIFICANCE

Land classification plays a critical role in urban planning, agriculture, environmental monitoring, and disaster management. Automating this process reduces manual effort and improves decision-making accuracy.

🛰️ Project Scope

✔ IN SCOPE

- Dataset selection and cleaning
- Image preprocessing and augmentation
- Model training and evaluation
- Performance analysis
- Documentation and presentation

⊘ OUT OF SCOPE

- Real-time satellite feeds
- Commercial deployment
- Multilingual interfaces

⚙ Methodology, Algorithms & Techniques

🧹

DATA PREPARATION

Data Cleaning, Preprocessing, and Exploratory Data Analysis (EDA)

FEATURE EXTRACTION

CNN-based feature extraction from RGB and multispectral imagery

MODEL TRAINING

Deep learning architectures (CNNs) with hyperparameter tuning

EVALUATION

Accuracy, Precision, Recall, F1-Score, and Confusion Matrices

---

🏆 Expected Results & Outcomes

Hypothesis: The trained model will achieve high classification accuracy across multiple land types.

- Accurate classification results across 10 land-use categories
- Clear performance metrics (Accuracy ≥ 80% target)
- Confusion matrix and detailed evaluation reports
- Well-documented, reproducible ML pipeline

👥 Task Division & Roles

Team Size: 6 Members — tasks divided equally across members.

TASK	ASSIGNED MEMBERS
Data Collection & Cleaning	Abdalaziz + Fatma
Model Development	Maram + Abdallah + Abdalrahman
Evaluation & Testing	Sarah + Abdalaziz + Fatma
Deployment & Visualisation	Maram + Abdallah + Abdalrahman
Documentation & Presentation	Each sub-team presents their phase

Roles rotate to ensure balanced contribution.

⚠ Risk Assessment & Mitigation

RISK	MITIGATION STRATEGY
Poor data quality	Early data inspection and validation pipeline

Model underperformance	Start with baseline models, iterate
Time constraints	Weekly progress reviews and milestone tracking

📉 Key Performance Indicators (KPIs)

≥80%
Model Accuracy Target

P & R
Precision & Recall

⏱
Training & Inference Time

✔
Milestone Completion Rate

📄 Content Organization & Documentation

All project phases are documented clearly and professionally. The final report includes methodology, analysis, results, and conclusions. The team is prepared to explain dataset choices, model selection rationale, evaluation metrics, and limitations & future work.

🗓 Detailed Timeline Aligned with Internship Deadlines

NOW – 20 FEBRUARY 2026 · 16 DAYS
Phase 1 — Project Planning & Management
Finalize project proposal · Assign roles and responsibilities · Risk assessment and KPI definition

21 FEBRUARY – 20 APRIL 2026 · 42 DAYS
Phase 2 — Literature Review & Requirements Analysis
Review related work in land classification · Dataset selection and understanding · Define functional and non-functional requirements

21 APRIL – 1 MAY 2026 · 9 DAYS
Phase 3 — System Analysis & Design
System architecture design · Data pipeline design · Model selection strategy

2 MAY – 10 JULY 2026 · 51 DAYS
Phase 4 — Implementation
Data cleaning and preprocessing · Model training and tuning · Evaluation and validation

11 JULY – 17 JULY 2026 · 6 DAYS
Phase 5 — Finalization & Presentation
Final report writing · Presentation preparation · Final system review

GANTT CHART OVERVIEW

A Gantt chart visualizes the project schedule, task dependencies, and parallel work across team members to ensure balanced workload distribution and progress tracking.

📊 PROJECT GANTT CHART — INTERNSHIP TIMELINE

PHASE	DURATION	DATES	PROGRESS BAR
Project Planning & Management	16 days	Feb 1 → Feb 20, '26	
Literature Review & Requirements	42 days	Feb 23 → Apr 21, '26	
System Analysis & Design	9 days	Apr 22 → May 4, '26	
Implementation (Data, Training, Eval)	51 days	May 5 → Jul 14, '26	
Finalization & Presentation	6 days	Jul 15 → Jul 22, '26	

PART II · ACADEMIC EVALUATION

2

🎓 Lecturer's Assessment

Assessment Summary: The project demonstrates a highly systematic approach to deep learning. The exhaustive Exploratory Data Analysis (EDA) on both standard RGB and 13-band multispectral data provides excellent context. The comparative modeling strategy—utilizing transfer learning for RGB and a custom-built ResNet-50 architecture tailored for 13-band TIF data—highlights strong architectural understanding.

💡 Suggested Improvements

CROSS-VALIDATION

Implement k-fold cross-validation instead of a single static train/val/test split to yield more robust performance metrics.

ADVANCED ARCHITECTURES

Explore Vision Transformers (ViTs) alongside ResNet to compare CNN performance against self-attention mechanisms.

INTERACTIVE DEMO

Integrate a lightweight web framework (e.g., Streamlit or Gradio) to allow users to upload custom satellite images for real-time inference.

Final Grading Criteria

Based on course syllabus — percentage breakdown per evaluation domain.

20%	Documentation & Requirements Analysis
40%	Implementation, Code Quality & Best Practices
20%	System Testing, Evaluation & Accuracy Metrics
20%	Final Presentation & Demo

Stakeholder Analysis

DATA SCIENTISTS / ML ENGINEERS

Require well-structured, clean, validated datasets and reproducible code pipelines that handle large volumes of data efficiently via batching, without memory overflow.

🌿

ENVIRONMENTAL RESEARCHERS / URBAN PLANNERS

Benefit from the final classification outputs — requiring high-accuracy results to monitor deforestation, track urbanization, and evaluate crop health.

📝

LECTURERS / ASSESSORS

Require transparent, well-commented code, logical system design, and comprehensive reports (confusion matrices, F1-scores) to fairly evaluate the project's success.

📖 User Stories & Use Cases

USER STORY 1 · RESEARCHER

As a researcher, I want to visualize the pixel intensity distributions of both RGB channels and Multispectral bands so that I can understand the underlying properties of different land-use categories.

USER STORY 2 · DATA SCIENTIST

As a data scientist, I want to train a custom ResNet-50 model specifically on 13-band TIF data to determine if multispectral data yields better classification accuracy than standard RGB imagery.

USER STORY 3 · ASSESSOR

As an assessor, I want the system to output confusion matrices and classification reports so that I can identify precisely which land-cover classes the model struggles to differentiate.

⚙ Functional Requirements

#	REQUIREMENT	DESCRIPTION
FR-01	**Data Ingestion & Validation**	The system must automatically detect the environment (Kaggle vs. Local), load RGB (.jpg) and Multispectral (.tif) images, and check for corrupted files, missing labels, and duplicate hashes.
FR-02	**EDA Generation**	The system must generate visual analytics including class distributions, channel histograms, HSV spatial analysis, and spectral signatures for 13 bands.
FR-03	**Data Pipelines**	The system must utilize `tf.data` generators to load images in batches (e.g., 32 images per batch), applying dynamic data augmentation to training sets to prevent overfitting.
FR-04	**Model Architecture**	The system must support transfer learning via a pretrained ResNet-50 model (RGB) and compile a custom 13-channel ResNet-50 from scratch (TIF).
FR-05	**Evaluation**	The system must evaluate models on an unseen test set and output accuracy, precision, recall, F1-scores, and plotted learning curves.

🛡 Non-Functional Requirements

CATEGORY	REQUIREMENT
Performance	Training algorithms must be optimized for GPU acceleration. Data loading should utilize background prefetching to ensure the GPU is never idle waiting for data.
Reliability	The data pipeline must handle exceptions gracefully, skipping unreadable or corrupted files rather than crashing the execution.
Reproducibility	Global random seeds (Seed: 42) must be strictly enforced across NumPy, Python, and TensorFlow to guarantee identical results across runs.
Maintainability & Usability	The codebase must be highly modular ("Chunks") and heavily commented, adhering to industry-standard naming conventions.

🔍 4.1 — Problem Statement & Objectives

Problem Statement: Classifying land cover from satellite imagery is a critical task for global environmental monitoring. However, traditional manual analysis is slow, and standard machine learning methods struggle with the sheer scale of the data and the high dimensionality of multispectral imagery.

OBJECTIVES

1. Build an automated deep learning pipeline capable of classifying 10 distinct land-use categories using the EuroSAT dataset.
2. Perform rigorous EDA to understand the statistical distribution of the dataset.
3. Compare the classification efficacy of standard RGB imagery against 13-band multispectral data.

USE CASE DESCRIPTION — LOGICAL ACTOR FLOW

ACTOR	ACTION	SYSTEM RESPONSE
System Operator	Executes the pipeline scripts	The system automatically locates data paths, validates splits, trains the required neural networks, monitors validation loss to trigger early stopping, and dumps evaluation artifacts to the working directory.

SOFTWARE ARCHITECTURE

The system adopts a **Sequential Data Processing Pipeline** architecture:

Data Validation → **EDA** → **tf.data Pipeline** → **Model Compilation** → **Training Loop** → **Evaluation & Reporting**

🗄 4.2 — Database Design & Data Modeling

Note: Because this is a Machine Learning pipeline, it relies on an organized flat-file data lake rather than a traditional relational database (RDBMS).

LOGICAL SCHEMA

LAYER	STRUCTURE & DESCRIPTION
Dataset Lake	Organized hierarchically by classes. `EuroSAT/` (RGB) and `EuroSATallBands/` (TIF), each containing 10 subdirectories (e.g., `Forest`, `Highway`).
Metadata Tables	`train.csv`, `validation.csv`, and `test.csv` acting as relational tables linking the `Filename` (Primary Key) to the `ClassName` and integer `Label`.
Artifact Storage	Output weights stored logically as `.keras` checkpoints. Metrics maintained in-memory as history dictionaries and saved as visualization plots.

4.3 — Data Flow & System Behavior

DATA FLOW DIAGRAM (DFD)

- Raw images and CSV tables flow into the **Data Validation Module**.
- Validated file paths flow into the **Data Generators**, where they are transformed into mathematical Tensors (normalized arrays).
- Tensors flow through the deep neural network (ResNet) to output classification probabilities.
- Ground truth labels and probabilities flow into the **Evaluation Module** to compute final loss and metrics.

SEQUENCE DIAGRAM

Operator → **Pipeline Script** → **Env Detector** → **Data Loader** → **Model Builder** → **GPU Engine** → **Evaluator** → **Operator**

ACTIVITY DIAGRAM

Start → **Check Environment** → **Validate Data** → **Generate EDA** → **Build tf.data.Dataset** → **Initialize Weights** → **Train Epochs** → **Early Stopping?** → **Evaluate on Test** → **Save Results** → **End**

CLASS DIAGRAM (CONCEPTUAL)

Although written procedurally, the logical objects include:

LOGICAL CLASS	ATTRIBUTES	METHODS
DatasetManager	paths, splits	load_csv, hash_check
EDAVisualizer	—	plot_histograms, plot_spectral_signatures
ModelArchitect	—	build_rgb_resnet, build_tif_resnet
Evaluator	—	plot_learning_curves, generate_classification_report

---

4.4 — UI/UX Design & Prototyping

Console & Notebook UI: As a data pipeline, the primary UI is the standard output (terminal/notebook output) and

generated figures. Matplotlib and Seaborn are used with the `darkgrid` style. Custom color mapping ensures the 10 distinct classes maintain the same assigned color across all bar charts, pie charts, and scatter plots.

FEEDBACK UI DESIGN

Print statements are structured with clear headers, horizontal dividers (====), and emojis (✅ ⚠ 📊 ⏱) to allow the user to easily scan the logs, understand the pipeline's progress, and immediately notice warnings (e.g., missing data or GPU absence).

🚀 4.5 — System Deployment & Integration

TECHNOLOGY STACK

Python 3 | TensorFlow & Keras | Pandas | NumPy | Rasterio | Matplotlib | Seaborn | ResNet-50 | tf.data API | Scikit-Learn

DEPLOYMENT DIAGRAM (LOGICAL)

The software is intended to be deployed on a cloud notebook environment (e.g., Kaggle or Google Colab).

NODE	ROLE
Hardware Node	Uses a CPU for data augmentation and a GPU (e.g., NVIDIA T4 or P100) for matrix multiplication and backpropagation.
Storage Node	Dataset is mounted as a read-only input drive, while model checkpoints and generated plots are directed to an ephemeral read-write working directory.

COMPONENT DIAGRAM — FOUR MAIN COMPONENTS

Ingestion Engine — Handles OS paths, CSV parsing, and Rasterio TIF reads. Responsible for environment detection and raw data loading.

Transformation Engine — Handles min-max normalization, ImageNet standardization, and augmentation. Transforms raw pixel arrays into model-ready tensors.

Machine Learning Core — Contains the ResNet-50 computational graphs for both the pretrained RGB model and the custom 13-band TIF model.

Reporting Engine — SciKit-Learn metrics calculator and Plotly/Matplotlib image generator for confusion matrices, learning curves, and classification reports.